

เอกสารข้อกำหนดความต้องการซอฟต์แวร์ (SRS)

BidNest — Auction & Marketplace

ภาพรวมโปรเจก

แพลตฟอร์มนี้เป็นระบบตลาดออนไลน์แบบ full-stack ที่ responsive รวม 2 รูปแบบการซื้อขายที่เสริมกันไว้ภายใต้ระบบบัญชีเดียว ได้แก่ โมดูล Auction (ประมูลออนไลน์แบบเรียลไทม์) และโมดูล E-commerce (ซื้อขายราคาคงที่) โดยผู้ใช้ 1 บัญชีสามารถเป็นทั้งผู้ประมูล ผู้ขายของประมูล ผู้ซื้อ และผู้ขายสินค้าได้พร้อมกัน

โมดูล Auction สามารถ สมัครสมาชิกหรือ login ผ่าน social, สร้างและเผยแพร่การประมูล, เข้าร่วม lobby ตามเวลาที่กำหนด, แข่งประมูลแบบเรียลไทม์, ใช้กลไก anti-sniping ต่อเวลาในการประมูล, สรุปผลว่า Sold หรือ Unsold, และแจ้งเตือนผู้ใช้ที่เกี่ยวข้อง

โมดูล E-commerce สามารถ ซื้อ/ขาย/เรียกดูหรือค้นหาตามหมวดหมู่, เพิ่มสินค้าลงตะกร้า, ทำการชำระเงินแบบจำลอง (simulated checkout) ที่จะแยกออกเคอร์ตามผู้ขายอัตโนมัติเมื่อจำเป็น, ติดตามสถานะจัดส่งแบบจำลองจนถึงปลายทาง, และดูประวัติคำสั่งซื้อได้ทั้งในมุมมองผู้ซื้อและผู้ขาย

ทั้งสองโมดูลใช้ระบบยืนยันตัวตนร่วมกัน — เข้าสู่ระบบด้วยอีเมล/รหัสผ่าน หรือผ่าน Google และ Line, ยืนยันตัวตนสองชั้น (2FA) ด้วยรหัส OTP ทางอีเมล, JWT access/refresh token, protected routes และสิทธิ์ USER/ADMIN — รวมถึงใช้ชุดหมวดหมู่ (category) ร่วมกันชุดเดียว ทั้งสองโมดูลยังใช้ Customer Service AI Chatbot ร่วมกันสำหรับช่วยตอบปัญหาการใช้งานเบื้องต้น

1. วัตถุประสงค์และขอบเขต

ระบบนี้เป็นตลาดออนไลน์ที่ใช้ผ่านเบราว์เซอร์ รองรับทั้ง desktop, tablet และ mobile โดยมี 2

รูปแบบการซื้อขายที่เกี่ยวข้องกันแต่ทำงานแยกอิสระจากกันภายใต้ระบบบัญชีเดียว คือ การประมูลแข่งราคา (Auction) และการซื้อขายราคาคงที่ (E-commerce) ระบบเปิดให้ค้นหา/เรียกดูข้อมูลได้แบบสาธารณะในทั้งสองส่วน ผู้ยืนยันตัวตนเท่านั้นจึงมีสิทธิ์ทำธุรกรรม ในเวอร์ชัน V1 นี้ไม่มีการโอนเงินจริงเกิดขึ้นในระบบ ส่วนการชำระเงินและจัดส่งฝั่ง e-commerce เป็นการจำลอง (simulated) ทั้งหมด ไม่ได้เชื่อมต่อกับผู้ให้บริการชำระเงินหรือขนส่งจริง

1.1 ขอบเขตที่รวมอยู่ในระบบ

- สมัครสมาชิกและเข้าสู่ระบบด้วยอีเมล/รหัสผ่าน, Google OAuth, Line OAuth; ยืนยันตัวตนสองชั้น (2FA) ด้วยรหัส OTP ทางอีเมลแบบบังคับทุกบัญชี โดยจัดการฝั่ง client ทั้งหมดผ่าน NextAuth (Auth.js) ในขณะที่ NestJS ยังคงเป็นเจ้าของ token ตัวจริง; JWT access token พร้อมระบบหมุนเวียน refresh-session และการเพิกถอนตอน logout; ป้องกันหน้าที่ต้อง login ทุกหน้าด้วย protected-route guard; ; ภูคิน รหัสผ่านที่ลิ้มผ่านลิงก์ทางอีเมล (AUTH-005)
- จัดการโปรไฟล์ผู้ใช้พร้อมที่อยู่จัดส่งเริ่มต้น, สิทธิ์ USER/ADMIN; ชุดหมวดหมู่ (category) เดียวที่ Admin จัดการ ใช้ร่วมกันทั้ง Auction และ E-commerce
- Auction: หน้าค้นหา/รายละเอียดแบบสาธารณะ, Hot Auctions, reserve แบบซ่อนได้ (ไม่บังคับ), กำหนด increment ขั้นต่ำ, ประวัติการประมูลเรียงตามเวลาแบบปิดบiddingผู้ประมูล, ขั้นตอน draft/preview/publish/edit/cancel, การประมูลแบบเรียลไทม์ที่มี transaction ผ่าน WebSocket room เฉพาะแต่ละรายการประมูล, บันทึกประวัติการต่อเวลาแบบ anti-sniping, สถานะวงจรชีวิตของการประมูล, watchlist, และ Live Arena ที่ responsive
- E-commerce: แดตตาล็อกสินค้าที่ค้นหา/กรองได้ตามหมวดหมู่ และราคา; ผู้ขายสร้าง/แก้ไข/ปิด การขายสินค้าของตัวเองได้ จัดการสต็อกได้ และสามารถตั้งส่วนลดอัตโนมัติตามจำนวนซื้อได้ ตะกร้าสินค้าที่เพิ่ม/แก้ไข/ลบได้; ระบบ checkout ที่แยกออกเคอร์ตามผู้ขายอัตโนมัติเมื่อตะกร้ามีสินค้าจากหลายผู้ขาย; การชำระเงินแบบจำลองผ่าน mock provider; การติดตามสถานะจัดส่งแบบจำลองพร้อม timeline; ประวัติคำสั่งซื้อทั้งฝั่งผู้ซื้อและผู้ขาย
- สิ่งที่ใช้ร่วมกัน: การแจ้งเตือนในแอปสำหรับผู้ที่ใช้ login แล้ว (ฝั่งประมูล: Outbid/Won/Ended/Cancelled, ฝั่งคำสั่งซื้อ: Placed/Shipment-Updated/Delivered), Customer Service AI Chatbot สำหรับช่วยตอบปัญหาการใช้งาน, การจัดการผู้ใช้/หมวดหมู่/รายการขายโดย Admin, การยกเลิกประมูลถูกเงินและการถอดสินค้าออกจากระบบ, บันทึก audit, เอกสาร API แบบ Swagger และ local environment ผ่าน Docker
- ฟีเจอร์เสริม (Optional): AI Price Estimator ช่วยแนะนำราคาเริ่มต้น/ราคาปิดประมูลให้ผู้ขายฝั่งประมูล (AI-002) และ AI Negotiator ให้ผู้ซื้อฝั่ง e-commerce เสนอราคาต่อรองได้ (AI-003) และสามารถตั้งราคาต่ำสุดที่รับได้แบบเป็นความลับ (negotiation floor); ผู้ซื้อสามารถเสนอราคาต่อรองผ่าน AI ได้ และการทบทวนระหว่างผู้ซื้อ-ผู้ขายต่อสินค้า (CHAT-001..003, NOT-008);

1.2 ขอบเขตที่ไม่รวมอยู่ในระบบ

- การเชื่อมต่อ payment gateway จริงในทั้งสองโมดูล; การคืนเงิน; การจ่ายเงินให้ผู้ขาย (payout) หรือบัญชี payout; ขั้นตอนแก้ไขข้อพิพาท; การยืนยันตัวตนผู้ขาย (seller verification); รีวิว/ให้คะแนนสินค้า และหน้าร้านของผู้ขาย (storefront)
- Live Arena แบบดูอย่างเดียสำหรับ Guest, การทำ Redis/horizontal scaling, แอป PWA/native, และการวิเคราะห์ข้อมูลเชิงลึกสำหรับผู้ขายหรือแพลตฟอร์ม
- Featured Auctions, การให้คะแนนความนิยม, การรายงานประมูลโดยผู้ใช้และคิวจัดการรายงานโดย Admin; การเชื่อมต่อกับผู้ให้บริการขนส่งจริง (เลขพัสดุและสถานะทั้งหมดสร้างขึ้นโดยระบบเอง ไม่ได้ดึงจาก API ของบริษัทขนส่งจริง); การส่งไฟล์รูปภาพในแชทซื้อ-ขาย (ข้อความตัวอักษรเท่านั้นใน V1)

2. ผู้ใช้งานระบบและสิทธิ์การเข้าถึง

ผู้ใช้งาน	คำจำกัดความ	สิ่งที่ทำได้	สิ่งที่ทำไม่ได้
Guest	สถานะที่ยังไม่ได้ login; <u>ไม่ใช่ role ที่บันทึกในระบบ</u>	เรียกดูรายการประมูลสถานะ Scheduled/Active/Sold/Unsold ที่เป็นสาธารณะ และสินค้าสถานะ ACTIVE ที่เป็นสาธารณะ; ดูรายละเอียดประมูล/สินค้าและประวัติการประมูลแบบปิดบังชื่อ; ค้นหา/กรองได้ทั้งสองโมดูล; สมัครสมาชิก/เข้าสู่ระบบ	สร้าง/ดูประมูล, ลงขายหรือซื้อสินค้า, เข้าร่วม Live Arena (ต้อง login), ประมูล, เพิ่มสินค้าลงตะกร้า/checkout, ขอประเมินราคาจาก AI, หรือเสนอราคาต่อรองผ่าน AI
User	role ที่บันทึกในระบบคือ USER โดย 1 บัญชีสามารถเป็นทั้งผู้ขายของประมูล ผู้ประมูล ผู้ขายสินค้า และผู้ซื้อได้พร้อมกัน	จัดการโปรไฟล์; สร้าง/จัดการประมูลของตัวเอง; watch ประมูล; เข้าร่วม Live Arena; ประมูลรายการของผู้อื่นที่อยู่ในสถานะ Active; ขอประเมินราคา AI สำหรับ draft ประมูลของตัวเอง; ลงขาย/แก้ไข/ปิดการขายสินค้า สต็อก และ negotiation floor ของตัวเอง; เพิ่มสินค้าลงตะกร้า; เสนอราคาต่อรองและ checkout (ยกเว้นสินค้าของตัวเอง); ติดตามคำสั่งซื้อ/การจัดส่งของตัวเอง; ดูประวัติ/การแจ้งเตือนของตัวเอง	ประมูลรายการที่ตัวเองสร้าง; เพิ่มสินค้าของตัวเองลงตะกร้าหรือซื้อสินค้าของตัวเอง; เข้าถึงฟังก์ชันของ Admin
Administrator	role ที่บันทึกในระบบคือ ADMIN	ดูแลติต; จัดการหมวดหมู่; ระบุ/เปิดใช้งานบัญชีผู้ใช้ใหม่; ยกเลิกประมูลที่ไม่เหมาะสม; ปิด/เปิดการขายสินค้าที่ไม่เหมาะสมกลับคืน; ดูภาพรวมคำสั่งซื้อ; ดูประวัติ audit	สร้างรายการประมูลหรือสินค้าในตลาด, ดูหรือประมูลในฐานะผู้เข้าร่วม, เพิ่มสินค้าลงตะกร้า, หรือ checkout

3. ข้อกำหนดทางเทคนิค

ส่วนของระบบ	เทคโนโลยี / ข้อกำหนดที่ต้องใช้
Web	Next.js App Router, React, TypeScript, Tailwind CSS, shadcn/ui, React Hook Form, Zod, TanStack Query
API	NestJS, TypeScript, REST API ที่มีเอกสารผ่าน Swagger, class-validator
Real-time	NestJS WebSocket gateway ที่มี room แยกตามแต่ละรายการประมูลสำหรับการประมูล และช่องทางสำหรับ push สถานะคำสั่งซื้อ/การจัดส่งให้ผู้ซื้อและผู้ขาย และ room แยกตามแต่ละการสนทนาซื้อ-ขายสำหรับข้อความแชทแบบเรียลไทม์ (CHAT-001..003)
AI features	มี 3 พีเจอรืที่กำหนดขอบเขตชัดเจน: (บังคับ) Customer Service AI Chatbot ใช้ LLM สนทนากับผู้ใช้แบบมีขอบเขตตอบจากฐานความรู้แพลตฟอร์มและข้อมูลของผู้ใช้คนนั้นเท่านั้น; (Optional) LLM ที่รับรูปภาพได้ (เช่น Claude หรือ GPT-4o แบบรับ image input) ใช้สำหรับ AI Price Estimator และ LLM หรือ LLM ผสม สำหรับ AI Negotiator สองพีเจอรืเสริมนี้คืนค่าผลลัพธ์แบบมีโครงสร้าง (function-called output) ไม่ใช้การสนทนาอิสระ
Data	PostgreSQL และ Prisma ORM; ใช้ database transaction เพื่อความถูกต้องของการประมูล การตัดสต็อก และการแยกออเดอร์ตามผู้ขายหลายราย

ส่วนของระบบ	เทคโนโลยี / ข้อกำหนดที่ต้องใช้
Identity	JWT, refresh session, อีเมล/รหัสผ่าน, Google OAuth, Line OAuth, การยืนยันตัวตนสองชั้นด้วย OTP ทางอีเมลแบบบังคับทุกบัญชี โดย NestJS เป็นผู้ออกและบันทึก access/refresh token ทุกตัว; การป้องกันหน้าที่ต้อง login (protected-route) ถูกบังคับใช้แยกกันทั้งฝั่ง Next.js middleware และฝั่ง NestJS guard
Auth orchestration (Next.js)	NextAuth (Auth.js): ใช้ provider สำเร็จรูปสำหรับ Google และ Line ในการ redirect OAuth และใช้ Credentials provider ครอบการ login ด้วยอีเมล/รหัสผ่าน + OTP โดย NextAuth
Email delivery	Maildev รันเป็นอีก service หนึ่งใน Docker Compose คู่กับ PostgreSQL คอยดักอีเมลทั้ง 2 ประเภทที่ระบบส่งในช่วง development — รหัส OTP (AUTH-007) และลิงก์รีเซ็ตรหัสผ่าน (AUTH-005) — ส่วน production ส่วน production จะส่งผ่าน SMTP relay จริง (เช่น Resend, SendGrid หรือ Nodemailer + SMTP) ผ่าน interface เดียวกัน
Payment (simulated)	โมดูลชำระเงินจำลองอยู่หลัง interface ที่ไม่ผูกกับผู้ใช้บริการรายใดรายหนึ่ง; บันทึกแถวข้อมูลใน payment_transactions ด้วยสถานะ SUCCEEDED/FAILED และไม่ติดต่อกับผู้ใช้บริการชำระเงินจริงหรือโอนเงินจริง
Deployment	Docker Compose ใช้สำหรับ infrastructure ตอน dev เท่านั้น คือ PostgreSQL และ Maildev ส่วนแอป Next.js และ NestJS รันตรงบนเครื่อง (native) ผ่าน pnpm เพื่อให้ hot-reload เร็ว ไม่ได้รันอยู่ใน container ส่วน shared contract เก็บไว้ที่ packages/contracts

4. ข้อกำหนดเชิงฟังก์ชัน (Functional Requirements)

4.1 การยืนยันตัวตน โปรไฟล์ และการควบคุมการเข้าถึง

ID	ข้อกำหนด	เกณฑ์การยอมรับ
AUTH-001	การสมัครสมาชิกแบบ Local	ต้องกรอกชื่อจริง, ชื่อที่แสดง (display name), อีเมลที่ไม่ซ้ำ และรหัสผ่าน; นามสกุลเป็นข้อมูลไม่บังคับ; รหัสผ่านถูก hash อย่างปลอดภัย; นอกจากนี้ ระบบจำ browser ที่เคยตอบรหัสถูกต้องแล้วได้ ถ้าผู้ใช้เลือก “จำอุปกรณ์นี้” ตอนกรอกรหัส การ login ครั้งถัดไปของบัญชีเดิมจาก browser เดิมจะผ่านโดยไม่ต้องขอรหัสอีก จนกว่าจะครบอายุที่ตั้งไว้ (ค่าเริ่มต้น 30 วัน กำหนดด้วย TRUSTED_DEVICE_TTL_DAYS) รหัสยังคงบังคับเสมอสำหรับการ login ครั้งแรกจาก browser ใหม่ ระบบเก็บเฉพาะค่า hash ของ token ประจำอุปกรณ์ ไม่เก็บตัว token และ token ที่เป็นของบัญชีอื่นต้องใช้เปิดบัญชีนี้ได้ อุปกรณ์ที่จำไว้ทั้งหมดต้องถูกลบเมื่อผู้ใช้รีเซ็ตรหัสผ่าน (AUTH-005) การจำอุปกรณ์แทนได้เฉพาะปัจจัยที่สละเท่านั้น ไม่แทนรหัสผ่าน ทุกครั้งที่ login ยังต้องตรวจรหัสผ่านตามปกติ ก่อนเสมอ อีกทางหนึ่ง ผู้ใช้สิทธิ์ ADMIN login ด้วยรหัสผ่านอย่างเดียวโดยไม่ต้องใช้รหัสจากอีเมลได้ เมื่อ environment นั้นเปิดสวิตช์ ADMIN_SKIP_2FA ไว้ ค่าเริ่มต้นของสวิตช์นี้คือปิด และมีไว้เพื่อความสะดวกของเครื่อง developer เป็นหลัก ผู้ใช้ทั่วไปไม่ได้รับการยกเว้นนี้ไม่ว่ากรณีใด และทุกครั้งที่ admin เข้าโดยไม่ใช้รหัสต้องบันทึก log
AUTH-002	การเข้าสู่ระบบแบบ Local	ผู้ใช้ที่ active กรอกอีเมลและรหัสผ่าน หากยังไม่ผ่าน AUTH-007 สำหรับการ login ครั้งนี้ ระบบจะตรวจสอบข้อมูล login แล้วส่ง OTP ทางอีเมล พร้อมส่งค่ากลับเป็นสถานะ “รอการยืนยัน” — ยังไม่ออก token ให้ ต้องส่งค่าขอครั้งที่สองพร้อมข้อมูล login เดิม บวกกับ OTP ที่ถูกต้องและยังไม่หมดอายุ ถึงจะได้รับ access token และ refresh session บัญชีที่ถูกระงับ/ปิดใช้งานจะถูกปฏิเสธตั้งแต่ขั้นตอนแรก
AUTH-003	Google OAuth	บัญชี Google หนึ่งบัญชีผูกกับบัญชีแพลตฟอร์มนี้เพียงบัญชีเดียวเท่านั้น
AUTH-004	Refresh Session	Refresh token ถูก hash, มีวันหมดอายุ และถูกเพิกถอนเมื่อ logout ข้อมูล session ที่บันทึกไว้มีแค่ ID, ผู้ใช้, hash, วันหมดอายุ, สถานะการเพิกถอน และเวลาที่สร้างเท่านั้น ส่วน session cookie ของ NextAuth เองจะถือสำเนา token คู่นี้ไว้ใช้งานฝั่ง browser เท่านั้น การ refresh จะเรียก endpoint เดียวกันของ NestJS เสมอ ดังนั้นข้อมูล session นี้ยังคงเป็นแหล่งข้อมูลจริงเพียงแหล่งเดียว ไม่ว่าฝั่งไหนจะอ่านค่าไปใช้ก็ตาม

ID	ข้อกำหนด	เกณฑ์การยอมรับ
AUTH-005	การกู้คืนรหัสผ่าน	ผู้ใช้กรอกอีเมลที่หน้า Forgot Password ระบบส่งลิงก์รีเซตรหัสผ่านแบบ single-use ไปยังอีเมลที่ลงทะเบียนไว้เท่านั้น (ไม่ยืนยันหรือปฏิเสธว่าอีเมลนี้มีอยู่ในระบบหรือไม่ เพื่อป้องกัน user enumeration) ลิงก์หมดอายุภายในเวลาที่กำหนด (เช่น 30 นาที) และใช้ได้ครั้งเดียว หลังตั้งรหัสผ่านใหม่สำเร็จ ทุก refresh session เดิมของบัญชีนี้จะถูกเพิกถอนทั้งหมดทันที (บังคับ login ใหม่ทุกอุปกรณ์) reset token ไม่ปรากฏใน API response หรือ production log เด็ดขาด
AUTH-006	Line OAuth	บัญชี Line หนึ่งบัญชีผูกกับบัญชีแพลตฟอร์มนี้เพียงบัญชีเดียว การ login ผ่าน Line ต้องการแค่ Line user ID เท่านั้น ส่วนอีเมลเป็นข้อมูลไม่บังคับเพราะ Line อาจไม่ให้ข้อมูลนี้มา การระบุตัวตนใช้คู่ provider + provider account ID เป็นหลัก การผูกบัญชีจะไม่เกิดขึ้นจากการจับคู่อีเมลที่ยังไม่ได้ยืนยันเพียงอย่างเดียวเด็ดขาด
AUTH-007	การยืนยันตัวตนสองชั้น (OTP ทางอีเมล, บังคับ)	การยืนยันตัวตนสองชั้นเป็นข้อบังคับสำหรับทุกบัญชี ทั้ง User และ Admin และบังคับใช้กับทุกวิธีการ login คือ local (AUTH-002), Google OAuth (AUTH-003) และ Line OAuth (AUTH-006) ทุกช่องทางต้องผ่านการยืนยัน OTP ทางอีเมลก่อนถึงจะออก session ให้ เนื่องจาก Credentials provider ของ NextAuth ตรวจสอบข้อมูลในการเรียก authorize() เพียงครั้งเดียว ผัง client จึงรวบรวมรหัสผ่านและ OTP เป็น 2 ขั้นตอนบนหน้าจอ แต่ส่งเข้ามาพร้อมกันในการเรียก signIn('credentials', ...) ครั้งเดียว ส่วน callback signIn ของ NextAuth ก็บังคับใช้ด้าน OTP เดียวกันนี้กับ Google/Line ด้วย เพื่อไม่ให้ provider ไหนข้ามขั้นตอนนี้ไปได้ รหัส 6 หลักแบบใช้ครั้งเดียวจะถูกส่งไปยังอีเมลที่ลงทะเบียนไว้ และต้องกรอกภายในเวลาที่กำหนด (เช่น 10 นาที) รหัสจะถูกยกเลิกทันทีที่ใช้สำเร็จหรือหมดเวลา; การขอส่งรหัสใหม่มีการจำกัดความถี่ (เช่น 1 ครั้งต่อ 60 วินาที) ส่วนรหัสสำรอง (backup/recovery code) ยังไม่อยู่ในขอบเขตของ V1
AUTH-008	Protected Routes (หน้าที่ต้อง Login)	ทุกหน้าที่ต้อง login (โปรไฟล์, ฟอรัมขาย/ลงประกาศ, ตะกร้า, checkout, ประวัติคำสั่งซื้อ, Live Arena, admin) ถูกป้องกันด้วย middleware ของ NextAuth ซึ่งอิงจาก session ใน AUTH-004 และยังคงตรวจสอบข้อมูลอย่างอิสระโดย NestJS guard ที่ตรวจสอบ access token ตัวเดียวกันทุกครั้งที่มีการเรียก request; หากเข้าถึงโดยไม่ได้อิน login จะถูก redirect ไปหน้า login
USR-001	โปรไฟล์ผู้ใช้	ผู้ใช้จัดการชื่อ/นามสกุล, ชื่อที่แสดง, รูปโปรไฟล์, bio, เบอร์โทร, ที่อยู่ และที่อยู่จัดส่งเริ่มต้นสำหรับ prefill ตอน checkout ผัง e-commerce ได้ ส่วนหน้าสาธารณะของประมวลและสินค้าจะแสดงเป็นชื่อที่แสดง (display name) หรือชื่อแบบปิดบัง

4.2 วงจรชีวิตของการประมวลและการค้นหา

วงจรชีวิตที่บันทึกในระบบ: DRAFT → SCHEDULED → ACTIVE → SOLD หรือ UNSOLD โดยสถานะ DRAFT และ SCHEDULED เปลี่ยนเป็น CANCELLED ได้ Preview เป็นแค่ขั้นตอนแสดงผลฝั่ง frontend เท่านั้น ส่วน Publish คือคำสั่งที่เปลี่ยนสถานะ draft ที่ถูกต้องให้กลายเป็นสถานะถัดไป

ID	ข้อกำหนด	เกณฑ์การยอมรับ
AUC-001	สร้าง Draft	ผู้ใช้สร้าง draft แบบส่วนตัว ประกอบด้วย ชื่อเรื่อง, รายละเอียด, หมวดหมู่ที่ active, สภาพสินค้า (ใหม่/มือสอง), ราคาเริ่มต้น, increment ขั้นต่ำ, reserve (ไม่บังคับ), กำหนดการ และรูปภาพ
AUC-002	ตรวจสอบก่อนเผยแพร่	ต้องกรอกชื่อเรื่อง, รายละเอียด, หมวดหมู่, สภาพสินค้า, เวลาเริ่ม/สิ้นสุด, ราคาเริ่มต้น และ increment; จำนวนเงินต้อง > 0; เวลาสิ้นสุดต้องอยู่หลังเวลาเริ่ม; ต้องมีรูปอย่างน้อย 1 รูป; reserve ต้อง ≥ ราคาเริ่มต้น
AUC-003	Reserve แบบซ่อน	ราคา reserve จะไม่เปิดเผยต่อสาธารณะเด็ดขาด จะส่ง/broadcast แค่ว่า reserveMet ที่คำนวณได้เท่านั้น ไม่มีการบันทึก reserve_met_at ลงฐานข้อมูล หากราคาประมูลต่ำกว่า reserve ผลลัพธ์จะเป็น UNSOLD
AUC-004	Preview และ Publish	ผู้ขาย preview หน้าตาที่ผู้ใช้จะเห็นได้โดยไม่เปลี่ยนสถานะ; draft ที่ผ่านการตรวจสอบแล้วจะ publish ออกมาเป็นสถานะ SCHEDULED หรือ ACTIVE
AUC-005	การมองเห็นตอน Scheduled	ประมวลที่อยู่ในสถานะ scheduled จะเป็นสาธารณะ ดูและเข้าร่วม lobby ได้ แต่จะเริ่มประมูลได้ก็ต่อเมื่อเข้าสู่สถานะ Active แล้วเท่านั้น

ID	ข้อกำหนด	เกณฑ์การยอมรับ
AUC-006	แก้ไข/ยกเลิก	ผู้ขายแก้ไข/ยกเลิกได้เฉพาะประมูลที่อยู่ในสถานะ Draft หรือ Scheduled เท่านั้น เมื่อเข้าสถานะ Active หรือมีการประมูลเกิดขึ้นแล้ว ข้อมูลหลักของประมูลจะแก้ไขไม่ได้อีก ยกเว้นการกระทำที่เกี่ยวกับการควบคุม/วงจรชีวิตของระบบ จะถูกจัดการโดย admin
AUC-007	การจบการประมูล	ราคาประมูลสูงสุดที่ถูกต้องและผ่าน reserve จะทำให้ผลลัพธ์เป็น SOLD และบันทึกผู้ชนะ/ราคาที่ชนะไว้; หากไม่มีการประมูลเลยหรือไม่ผ่าน reserve ผลลัพธ์จะเป็น UNSOLD
AUC-008	Hot Auctions	ประมูลที่ active และยังไม่ถูกลบ จะเรียงลำดับตามจำนวนบิตที่ยอมรับแล้วจากมากไปน้อย ตามด้วยเวลาสิ้นสุดปัจจุบันจากน้อยไปมาก แล้วตามด้วย auction ID โดยไม่มี flag พิเศษหรือ hidden scoring ใดๆ

4.3 การประมูล, Watchlist, การแจ้งเตือน และ Live Arena

ID	ข้อกำหนด	เกณฑ์การยอมรับ
BID-001	การประมูลที่ถูกต้อง	ต้องเป็นประมูลที่ active; ผู้ประมูลต้องเป็น USER ที่ active; ต้องไม่ใช่ผู้ขาย; จำนวนเงินต้อง $\geq$ ราคาปัจจุบัน + increment; ค่าขอต้องไม่ซ้ำ โดยจะบันทึกเฉพาะบิตที่ผ่านการยอมรับเท่านั้น
BID-002	Atomicity และ Idempotency	การตรวจสอบ, สร้างบิต, อัปเดตราคา/จำนวนบิตปัจจุบัน, คำนวณสถานะ reserve และต่อเวลา ทั้งหมดนี้ทำแบบ atomic; client request ID ใช้ป้องกันการส่งซ้ำจากการ retry
BID-003	อัปเดตการประมูลแบบเรียลไทม์	บิตที่ยอมรับแล้วจะถูก broadcast ใน room พร้อมสถานะสาธารณะที่ถูกต้องและสถานะ reserve ที่คำนวณได้ โดยจะไม่ส่งค่า reserve จริงออกไปเด็ดขาด
BID-004	Anti-sniping	หากมีการประมูลเข้ามาในช่วง 2 นาทีสุดท้าย จะต่อเวลาออกไปอีก 2 นาที ต่อได้สูงสุด 5 ครั้ง โดยบันทึกการต่อเวลาทุกครั้งไว้
BID-005	ประวัติการประมูล	ประวัติจำนวนเงิน/เวลาซื้อผู้ประมูลแบบปิดบัง เรียงตามเวลา โดยเคารพสิทธิ์ความเป็นส่วนตัว
WAT-001/002	Watchlist	ผู้ใช้ที่ login แล้วเพิ่ม/นำประมูลสาธารณะออกจาก watchlist ได้ครั้งละ 1 รายการ และเห็นสถานะ, เวลานับถอยหลัง, ราคาปัจจุบัน และผลลัพธ์ นอกจากรายการประมูลแล้ว ผู้ใช้กดติดตามสินค้าในร้าน (product) ได้ด้วยกลไกเดียวกัน โดยเก็บแยกตารางของตัวเองและใช้ composite key user_id + product_id เช่นเดียวกัน กดติดตามซ้ำต้องไม่เกิดรายการซ้ำและไม่ถือเป็นข้อผิดพลาด โดยยังคงวันที่ ติดตามครั้งแรกไว้เพื่อไม่ให้ลำดับในรายการสลับไปมา ยกเลิกติดตามสิ่งที่ไม่ได้ติดตามอยู่ก็ไม่ถือเป็น ข้อผิดพลาดเช่นกัน ติดตามได้เฉพาะสินค้าที่เผยแพร่เท่านั้น คือสถานะชุดเดียวกับที่ผู้ซื้อทั่วไปมองเห็น ในหน้าค้นหา และหน้ารายการที่ติดตามแสดงทั้งรายการประมูลและสินค้า โดยแบ่งหน้าแบบเดียวกับ หน้าค้นหา
NOT-001..004	การแจ้งเตือนฝั่งประมูล	การแจ้งเตือน Outbid, Auction Won, Auction Ended และ Auction Cancelled ที่บันทึกไว้ในระบบ ส่งผ่านแอป/WebSocket สำหรับผู้ใช้ที่ login แล้ว
LIV-001	Lobby ก่อนเริ่มประมูล	แสดงข้อมูลประมูล, จำนวนผู้เข้าร่วมแบบเรียลไทม์, เวลานับถอยหลัง, สถานะการเข้าร่วม และเปลี่ยนสถานะเริ่มประมูลอัตโนมัติ
LIV-002	Arena ขณะประมูล	แสดงราคาปัจจุบัน, ผู้นำแบบปิดบังชื่อ, บิดล่าสุด, เวลาที่เหลือ, ราคาต่ำสุดที่ประมูลต่อได้, สถานะ reserve-met และปุ่มควบคุมการประมูล
LIV-003/004/005	Sudden death, ผลลัพธ์, และความสมบูรณ์ของ UI	แสดงจำนวนครั้งที่ต่อเวลา/เวลาสิ้นสุดใหม่ในสถานะเร่งด่วนที่เข้าถึงง่าย; เมื่อจบแสดงผล Sold/Unsold/ราคาสุดท้าย; แอนิเมชันต้องไม่หน่วงเวลาหรือบดบังการประมูล

4.4 แคตตาล็อกสินค้าและการจัดการสินค้า (E-commerce)

แคตตาล็อกสินค้าใช้ชุดหมวดหมู่ร่วมกัน (ADM-003) แทนที่จะแยกชุดต่างหาก ทำให้ Admin จัดการรายการหมวดหมู่แค่ชุดเดียวสำหรับทั้งสองโมดูล แทนที่จะต้องจัดการสองชุด

ID	ข้อกำหนด	เกณฑ์การยอมรับ
PROD-001	สร้าง/ลงขายสินค้า	ผู้ขายสร้างรายการสินค้าประกอบด้วย ชื่อ, รายละเอียด, หมวดหมู่ที่ active, ราคา, จำนวนสต็อก, สภาพสินค้า (ใหม่/มือสอง) และรูปอย่างน้อย 1 รูป การลงขาย (ไม่มีขั้นตอน draft/schedule เพราะโมดูลนี้เป็นราคาคงที่ไม่ได้มีวงจรชีวิตแบบกำหนดเวลา) ต้องกรอกครบทุกช่องและสต็อก ≥ 0 จึงจะลงขายได้ทันทีในสถานะ ACTIVE
PROD-002	แก้ไข/ลบสินค้าของตัวเอง	ผู้ขายแก้ไขราคา รายละเอียด รูปภาพ หมวดหมู่ และสต็อกได้ในขณะที่สินค้าอยู่ในสถานะ ACTIVE หรือ INACTIVE และสลับสถานะระหว่าง ACTIVE กับ INACTIVE เองได้ การลบเป็นแบบ soft-delete ไปสถานะ REMOVED ซึ่งจะซ่อนจากการค้นหาสาธารณะ สินค้าที่ถูกอ้างอิงโดยคำสั่งซื้อที่ยังไม่ถูกยกเลิก อย่างน้อย 1 รายการ จะลบถาวรไม่ได้ ทำได้แค่ปิดการขาย เพื่อรักษาความถูกต้องของประวัติคำสั่งซื้อ สถานะ REMOVED เป็นสถานะปลายทาง ไม่มีการกู้คืนใน V1 หากผู้ขายต้องการเพียงหยุดขายชั่วคราว ให้ใช้ INACTIVE แทน ซึ่งเปิดกลับเองได้ตลอด สินค้าที่อยู่ในสถานะ SUSPENDED (ADM-005) ผู้ขายแก้ไขข้อมูล เปลี่ยนสถานะ หรือ soft-delete ไม่ได้ ทุกคำขอจะถูกปฏิเสธจนกว่า Admin จะเปิดการขายกลับให้ ที่ต้องห้าม soft-delete ด้วย เพราะการย้ายไป REMOVED จะลบร่องรอยว่าสินค้าเคยถูกระงับ ทำให้การตรวจสอบในขั้นถัดไปไม่เจอ
PROD-003	ค้นหาและเรียกดู	Guest หรือ User ค้นหาสินค้าสาธารณะที่สถานะ ACTIVE ได้ด้วยคำค้น, หมวดหมู่ (เลือกได้มากกว่า 1), และช่วงราคา; ผลลัพธ์เรียงตามราคาน้อย-มาก/มาก-น้อย หรือใหม่สุดก่อนได้; แสดงผลแบบแบ่งหน้า
PROD-004	รายละเอียดสินค้า	หน้ารายละเอียดสาธารณะแสดงราคา, จำนวนสต็อกคงเหลือแบบเรียลไทม์, ชื่อผู้ขายที่แสดง, หมวดหมู่, รูปภาพ และรายละเอียด; หากสถานะเป็น OUT_OF_STOCK ปุ่ม Add to Cart จะถูกปิดใช้งาน
PROD-005	การจัดการสต็อก	ผู้ขายตั้ง/แก้ไขจำนวนสต็อกได้จากหน้าจัดการสต็อกโดยเฉพาะ สต็อกจะถูกตัดก็ต่อเมื่อการชำระเงินสำเร็จ (SUCCEEDED) เท่านั้น (ไม่ตัดตอนเพิ่มลงตะกร้า) โดยทำใน transaction เดียวกับการสร้างคำสั่งซื้อ เพื่อป้องกันการขายเกินสต็อกเมื่อมีการ checkout พร้อมกันหลายคน สินค้าจะเปลี่ยนเป็น OUT_OF_STOCK อัตโนมัติเมื่อสต็อกเหลือ 0 และกลับเป็น ACTIVE เมื่อเติมสต็อกมากกว่า 0 แต่จะไม่สามารถเปลี่ยนสถานะสินค้าที่ถูก SUSPEND ได้ (ADM-005)
PROD-006	Negotiation Floor (ราคาต่ำสุด) (Optional)	ผู้ขายสามารถตั้งราคาต่ำสุดที่ยอมรับได้แบบเป็นความลับต่อสินค้าแต่ละชิ้นได้ที่ PROD-001/PROD-002 ราคานี้จะไม่ถูกเปิดเผยให้ผู้ซื้อเห็นหรือส่งกลับใน API response ผังผู้ซื้อเลย — ใช้กฎเดียวกับการปิดบัง reserve ของฝั่งประมูล (AUC-003) หากไม่ได้ตั้งราคานี้ไว้ ปุ่ม “เสนอราคา” (Make an offer) จะถูกซ่อนสำหรับสินค้านั้น และ AI-003 (พีเจอร์เสริม Optional) จะถูกปิดใช้งานไปด้วย
PROD-007	ส่วนลดอัตโนมัติตามจำนวนซื้อ	ผู้ขายตั้งค่าส่วนลดตามจำนวนต่อสินค้าได้ที่ PROD-001/PROD-002 (เช่น ซื้อตั้งแต่ N ชิ้นขึ้นไป ลด X% หรือ Y บาทต่อชิ้น) ส่วนลดคำนวณอัตโนมัติตอนแสดงยอดในตะกร้า/checkout จากกฎที่ตั้งไว้(ไม่ใช่ AI) กรณีที่ใช้ AI ต่อรองราคา จะไม่ได้ส่วนลดเพิ่มเติมจากการลดราคาปกติตามโปรโมชั่น(ไม่ทับซ้อนกัน) หากตั้ง negotiation floor (PROD-006) ไว้ต่ำกว่าราคาต่อชิ้นหลังหักส่วนลด(อ้างอิงจากราคาโปรโมชั่น) ระบบเตือนผู้ขายทันทีตอนตั้งค่า ว่าผู้ซื้อที่ต่อรองอาจได้ราคาต่ำกว่าคนที่ซื้อครบตามเงื่อนไขโปรโมชั่น ให้ผู้ขายตัดสินใจเองว่าจะปรับหรือไม่

4.4b การแชทระหว่างผู้ซื้อ-ผู้ขาย (Buyer-Seller Chat)

ID	ข้อกำหนด	เกณฑ์การยอมรับ
CHAT-001	เริ่มการสนทนา (Optional)	ผู้ใช้ที่ login แล้ว ยกเว้นผู้ขายสินค้าตัวเอง เปิดการสนทนากับผู้ขายได้จากหน้ารายละเอียดสินค้า (PROD-004) การสนทนาผูกกับคู่ product_id และ buyer_id, seller_id หนึ่งชุดต่อหนึ่ง การสนทนา ถ้าเคยเปิดแชทกับผู้ขายรายนี้สำหรับสินค้านี้แล้ว จะกลับไปใช้การสนทนาเดิม ไม่สร้างซ้ำ
CHAT-002	ส่งและรับข้อความ (Optional)	ผู้เข้าร่วมทั้งสองฝ่าย (buyer, seller) ส่งข้อความตัวอักษรได้ (ไม่รองรับไฟล์รูปภาพใน V1) ข้อความส่งผ่าน WebSocket แบบเรียลไทม์เข้า room เฉพาะของการสนทนานั้น (รูปแบบเดียวกับ room ของประมูลใน BID-003) ฝ่ายอื่นนอกเหนือจาก buyer/seller คู่นั้นไม่สามารถเข้าถึงเนื้อหาการสนทนา
NOT-008	แจ้งเตือน New Message (Optional)	ทุกครั้งที่ม่ข้อความใหม่ในการสนทนา (CHAT-002) ฝ่ายที่ไม่ได้เป็นผู้ส่ง (อีกฝ่ายในคู่สนทนา) จะได้รับการแจ้งเตือน New Message ทันที บันทึกไว้ในระบบและส่งผ่านแอป/WebSocket สำหรับผู้ใช้ที่ login แล้ว รูปแบบเดียวกับการแจ้งเตือนอื่นในระบบ (NOT-001..007)
CHAT-003	รายการสนทนาและการแจ้งเตือน (Optional)	ผู้ใช้ดูรายการการสนทนาทั้งหมดของตัวเอง (ฝั่งซื้อและฝั่งขายรวมกัน) พร้อมข้อความล่าสุดและจำนวนที่ยังไม่อ่าน การแจ้งเตือนข้อความใหม่ดู NOT-008
CHAT-004	แชทฝั่งประมูล และข้อความตอบกลับอัตโนมัติของผู้ขาย (Optional)	ผู้ซื้อกดปุ่มสอบถามผู้ขายจากหน้ารายละเอียดประมูล เพื่อเปิดห้องสนทนากับผู้ขายของรายการนั้นได้ เช่นเดียวกับที่ CHAT-001 ให้อำนาจกับสินค้าในร้าน ห้องสนทนา 1 ห้องผูกกับสินค้าอย่างเดียหรือประมูลอย่างเดียเสมอ ไม่ผูกทั้งสองอย่างพร้อมกัน และการเปิดซ้ำกับคู่สนทนาเดิมของประมูลเดิมต้องกลับไปห้องเดิมทุกครั้ง โดยบังคับด้วย unique auction_id + buyer_id + seller_id เปิดห้องได้เฉพาะประมูลที่เผยแพร่แล้วในสถานะที่ผู้ซื้อทั่วไปมองเห็น ผู้ขายเปิดห้องคุยกับรายการของตัวเองไม่ได้ การส่งข้อความการแจ้งเตือน New Message (NOT-008) และการจำกัดสิทธิ์ให้เห็นเฉพาะคู่สนทนาสองคน ใช้กติกาเดียวกับ CHAT-002 ทุกประการ นอกจากนี้ ผู้ขายตั้งข้อความตอบกลับอัตโนมัติของร้านไว้ล่วงหน้าได้ 1 ข้อความ ยาวไม่เกิน 500 ตัวอักษร เมื่อคำสั่งซื้อของผู้ซื้อรายใดชำระเงินสำเร็จเป็นสถานะ PAID ระบบจะส่งข้อความนั้นเข้าห้องสนทนาของสินค้าในคำสั่งซื้อนั้นให้อัตโนมัติในนามผู้ขาย 1 ครั้งต่อ 1 คำสั่งซื้อ เพราะคำสั่งซื้อ 1 ใบเป็นของผู้ขายรายเดียวอยู่แล้วตาม CART-003 ถ้าผู้ขายไม่ได้ตั้งข้อความไว้ หรือลบข้อความทิ้งโดยบันทึกเป็นคำว่าว่าง จะไม่มีการส่งข้อความใดๆ และไม่ถือเป็นข้อผิดพลาด การส่งข้อความอัตโนมัติที่ล้มเหลวต้องไม่ทำให้การชำระเงินที่สำเร็จไปแล้วล้มเหลวตาม และต้องไม่แสดงข้อผิดพลาดใดๆ ให้ผู้ซื้อเห็น ส่วนคำสั่งซื้อที่มาจากการชนะประมูลตาม CART-004 จะไม่ส่งข้อความอัตโนมัติ เพราะข้อความนี้ผูกกับห้องสนทนาของสินค้า และผู้ชนะประมูลมีห้องสนทนากับผู้ขายอยู่แล้วจากตอนประมูล

4.5 ตะกร้าสินค้าและการชำระเงิน (จำลอง)

ID	ข้อกำหนด	เกณฑ์การยอมรับ
CART-001	เพิ่มลงตะกร้า	ผู้ใช้ที่ login แล้ว ยกเว้นผู้ขายสินค้าตัวเอง สามารถเพิ่มสินค้าและจำนวนลงตะกร้าแบบถาวรของตัวเองได้ โดยจำนวนที่เพิ่มได้ถูกจำกัดตามสต็อกปัจจุบัน, เพิ่มลงตะกร้าได้เฉพาะ ACTIVE ถ้าของในตะกร้าเปลี่ยนสถานะที่หลัง checkout ไม่ได้
CART-002	จัดการตะกร้า	ผู้ใช้แก้ไขจำนวนหรือลบรายการสินค้าในตะกร้าได้ โดยยอดรวมในตะกร้าจะคำนวณใหม่จากราคาสินค้าปัจจุบันทุกครั้งที่แสดงผล ไม่ได้ตรึงราคาไว้จนกว่าจะ checkout ยอดนี้รวมส่วนลดอัตโนมัติตามจำนวนแล้ว ถ้าสินค้านั้นเข้าเงื่อนไข PROD-007 นอกจากแก้จำนวนและลบรายการแล้ว ผู้ใช้ยังดัดเลือกหรือยกเลิกการเลือกรายการเพื่อกำหนดว่าจะชำระเงินสำหรับรายการใดได้ด้วย (ดู CART-003) การไม่เลือกไม่ใช้การลบ รายการนั้นยังอยู่ในตะกร้าตามเดิม
CART-003	แยกคำสั่งซื้อตามผู้ขายหลายราย	ผู้ซื้อเลือกได้ว่าจะชำระเงินสำหรับรายการใดในตะกร้าบ้าง โดยค่าเริ่มต้นคือเลือกไว้ทุกรายการ ตอน checkout ระบบจะจัดกลุ่มเฉพาะรายการที่เลือกตามผู้ขาย และสร้าง Order แยกให้แต่ละผู้ขาย 1 รายการ (มีเลขออเดอร์ ยอดรวมย่อย และสถานะเป็นของตัวเอง) ภายใต้ checkout_session_id เดียวกัน ดังนั้นการ checkout ครั้งเดียวที่มีสินค้าจาก 3 ผู้ขาย จะได้ 3 ออเดอร์ที่ติดตามแยกจากกันได้ แต่ยัง

ID	ข้อกำหนด	เกณฑ์การยอมรับ
		จ่ายเงินครั้งเดียวรวมกันทั้งหมดที่เลือก (ดู CART-004) รายการที่ไม่ได้เลือกจะยังคงอยู่ในตะกร้าและไม่ได้ถูกตัดสต็อก ยอดรวมและจำนวนร้านที่แสดงบนหน้าตะกร้าและหน้าชำระเงินต้องคำนวณจากรายการที่เลือกเท่านั้น หากไม่ได้เลือกรายการใดเลยจะกด checkout ไม่ได้ และหากรายการที่เลือกไว้ถูกเขาออกจากตะกร้าไปแล้ว (เช่น จากอีกแท็บหนึ่ง) ระบบต้องปฏิเสธการชำระเงินทั้งครั้งแทนที่จะจ่ายเฉพาะส่วนที่เหลือ
CART-004	การชำระเงินแบบจำลอง	ผู้ซื้อเลือกวิธีชำระเงินจำลอง (บัตร / โอนผ่านธนาคาร / e-Wallet — เป็นแค่ตัวเลือกใน UI เท่านั้น ไม่มีการเก็บหรือส่งข้อมูลบัตรเครดิตใดๆ) และยืนยันที่อยู่จัดส่ง (ค่าเริ่มต้นมาจากที่อยู่ในโปรไฟล์ตาม USR-001 แกะไขได้ในแต่ละครั้งที่ checkout) ระบบ MockPaymentProvider จะคืนค่า SUCCEEDED หรือ FAILED กลับมา ระบบเรียก MockPaymentProvider เพียงครั้งเดียวต่อ checkout_session_id (ไม่ใช่ครั้งละ order) และบันทึกผลลัพธ์เป็น payment_transaction แยกเดียว หากเป็น SUCCEEDED ออกเดอร์ทั้งหมดในเชสชันนั้นจะเปลี่ยนเป็น PAID แต่ละออกเดอร์จะมีบันทึก snapshot ที่อยู่ของตัวเองลงใน order_addresses และตัดสต็อกแบบ atomic ที่ละรายการ (PROD-005) หากเป็น FAILED จะไม่มีการสร้างออกเดอร์ใดๆ และตะกร้าจะยังคงอยู่เหมือนเดิมทั้งรายการที่เลือกและไม่ได้เลือก นอกจากการชำระเงินจากตะกร้าแล้ว ผู้ชนะประมูลชำระค่ารายการที่ชนะผ่านหน้าชำระเงินเดียวกันนี้ได้ โดยส่ง auction_id มาแทน cart_item_ids และส่งทั้งสองอย่างพร้อมกันไม่ได้ เพราะเป็นการระบุสิ่งที่ต่ากันว่ากำลังจะซื้ออะไร ราคาที่เรียกเก็บมาจาก sold_price ของประมูลนั้นเท่านั้น ไม่รับตัวเลขราคาจากฝั่ง client เช่นเดียวกับที่ราคาสินค้าในตะกร้าไม่รับจาก client และผู้ที่ชำระได้ต้องเป็น winner_user_id ของประมูลนั้นเท่านั้น ประมูลที่ยังไม่จบ หรือจบแบบไม่ได้ขาย คือสถานะไม่ใช่ SOLD ชำระไม่ได้ ระบบต้องตอบข้อความปฏิเสธเหมือนกันทั้งกรณีที่ไม่มีการประมูลอยู่จริงและกรณีที่ประมูลนั้นไม่ใช่ของผู้ขอ เพื่อไม่ให้ผู้ที่ถือ id อยู่เดาได้ว่ารายการนั้นมีอยู่จริงหรือขายไปแล้วหรือไม่ ประมูล 1 รายการต้องชำระเงินได้ครั้งเดียวตลอดไป บังคับด้วย unique index บน order_items.auction_id เพื่อกันการจ่ายซ้ำจากการเปิดลิงก์ชำระเงินซ้ำหรือการกดพร้อมกันสองครั้ง กรณีที่ระบบตรวจพบว่ามีการชำระไปแล้วต้องปฏิเสธก่อนเรียกเก็บเงิน และหากสองคำขออนกันพอดีจนหลุดไปถึงชั้นฐานข้อมูล ต้องตอบเป็นความขัดแย้งพร้อม checkout_session_id ให้ไปตรวจสอบย้อนหลังได้ ไม่ใช่ข้อผิดพลาดทั่วไป ขั้นตอนที่เหลือทั้งหมดหลังจากนั้น ทั้งการเรียก MockPaymentProvider การบันทึก payment_transactions การสร้าง Order สถานะ PAID การ snapshot ที่อยู่ลง order_addresses การสร้าง Shipment และการแจ้งเตือน ใช้เส้นทางเดียวกับการชำระจากตะกร้าทุกประการ และอยู่ภายใต้ checkout_session_id ชุดเดียวกัน
CART-005	ยืนยันคำสั่งซื้อ	เมื่อสถานะเป็น PAID ผู้ซื้อและผู้ขายแต่ละรายจะได้รับการแจ้งเตือน Order Placed (NOT-005) ยอดรวมคำสั่งซื้อ, รายการสินค้า และ snapshot ที่อยู่ใน order_addresses จะแก้ไขไม่ได้อีกหลังชำระเงินสำเร็จ คำสั่งซื้อที่มาจากการชนะประมูลตาม CART-004 ต้องแสดงชื่อรายการประมูลที่ชนะและราคาปิดที่ต้องชำระ ทั้งบนหน้าสรุปก่อนจ่ายและบนใบเสร็จหลังจ่าย เพื่อให้ผู้ซื้อเห็นชัดว่ากำลังจ่ายค่าอะไร ไม่ใช่แสดงเป็นรายการสินค้าที่ว่างเปล่าเพราะบรรทัดนั้นไม่มี product_id
NOT-005	แจ้งเตือน Order Placed	เมื่อคำสั่งซื้อเข้าสู่สถานะ PAID (CART-005) ผู้ซื้อและผู้ขายแต่ละรายในเชสชันนั้นจะได้รับการแจ้งเตือน Order Placed ทันที บันทึกไว้ในระบบและส่งผ่านแอป/WebSocket สำหรับผู้ใช้ที่ login แล้ว รูปแบบเดียวกับการแจ้งเตือนฝั่งประมูล (NOT-001..004)

4.6 การติดตามการจัดส่งและประวัติคำสั่งซื้อ

ID	ข้อกำหนด	เกณฑ์การยอมรับ
SHIP-001	ผู้ขายอัปเดตสถานะจัดส่ง	ผู้ขายเปลี่ยนสถานะการจัดส่งของแต่ละออเดอร์ตัวเองไปตามลำดับที่กำหนดไว้โดยตัว: PROCESSING → SHIPPED → IN_TRANSIT → DELIVERED หรือจะ CANCELLED ก่อนถึงขั้น SHIPPED ก็ได้ เมื่อเข้าสู่สถานะ SHIPPED ระบบจะสร้างเลขพัสดุและชื่อขนส่งจำลองให้อัตโนมัติ สถานะจะย้อนกลับไม่ได้ ทุกการเปลี่ยนสถานะจะถูกบันทึกเพิ่มเข้าไปใน shipment_events (ผู้กระทำเป็นค่าว่างได้สำหรับขั้นตอนที่ระบบสร้างเอง ไม่มีการเก็บ JSON payload) ใช้รูปแบบ append-only เกี่ยวกับที่ใช้ใน auction_events อยู่แล้ว
SHIP-002	ผู้ซื้อติดตามการจัดส่ง	ผู้ซื้อดู timeline สถานะพร้อมเวลาของแต่ละออเดอร์ได้ โดยอ่านข้อมูลจาก shipment_events (การแจ้งเตือนที่เกี่ยวข้องดู NOT-006/NOT-007) สถานะ DELIVERED ถือเป็นสถานะสุดท้ายที่สำเร็จ — ใน V1 ยังไม่มีขั้นตอนการคืนสินค้า/คืนเงิน
SHIP-003	ประวัติคำสั่งซื้อ	ผู้ซื้อดูคำสั่งซื้อทั้งหมดของตัวเองจากทุกผู้ขาย พร้อมสถานะ, สรุปรายการสินค้า, ยอดรวม และวันที่กรรองตามสถานะได้ ผู้ขายก็ดูออเดอร์ทั้งหมดที่ได้รับสำหรับสินค้าของตัวเอง ด้วยตัวกรองแบบเดียวกัน ทั้งสองฝั่งแสดงผลแบบแบ่งหน้า
NOT-006	แจ้งเตือน Shipment Update	ทุกครั้งที่สถานะการจัดส่งเปลี่ยน (SHIP-001) ผู้ซื้อของออเดرنั้นจะได้รับการแจ้งเตือน Shipment Update ทันที อ่านสถานะปัจจุบันได้จาก shipment_events เกี่ยวกับที่ SHIP-002 ใช้แสดง timeline
NOT-007	แจ้งเตือน Delivered	เมื่อสถานะจัดส่งเข้า DELIVERED (สถานะสุดท้ายตาม SHIP-001) ผู้ซื้อจะได้รับการแจ้งเตือน Delivered เพิ่มเติมจาก Shipment Update ปกติ (NOT-006) เพื่อให้สังเกตได้ชัดว่าออเดอร์เสร็จสมบูรณ์แล้ว

4.7 AI Features: Customer Service Chatbot, Price Estimator และ Negotiator

ระบบมีพีเจอาร์ AI ทั้งหมด 3 ส่วน แบ่งเป็นพีเจอาร์บังคับ 1 ส่วนและพีเจอาร์เสริม (Optional) 2 ส่วน:

Customer Service AI Chatbot (AI-001) เป็นพีเจอาร์บังคับที่ช่วยตอบปัญหาการใช้งานเบื้องต้นให้ผู้ใช้ทุกคน ส่วน AI Price Estimator (AI-002) และ AI Negotiator (AI-003) ทั้งสองยังคงราคาต่ำสุดของผู้ขายเป็นความลับด้วยรูปแบบการปิดบังข้อมูลแบบเดียวกับ reserve ที่ซ่อนไว้ในฝั่งประมูล

ID	ข้อกำหนด	เกณฑ์การยอมรับ
AI-001	Customer Service AI Chatbot	ผู้ใช้ที่ login แล้วเปิด chat widget ในแอปได้ตลอดเวลาเพื่อถามปัญหาการใช้งานเบื้องต้น เช่น วิธีสร้าง/ประกาศประมูล, วิธีเพิ่มสินค้าลงตะกร้าหรือ checkout, การเช็คสถานะคำสั่งซื้อ/การจัดส่งของตัวเอง, วิธีเปิดใช้งาน 2FA ฯลฯ LLM ตอบคำถามจากฐานความรู้/FAQ ของแพลตฟอร์มและข้อมูลของผู้ใช้คนนั้นเอง (เช่น ดึงสถานะออเดอร์ผ่าน endpoint ที่มีอยู่แล้ว) เท่านั้น ไม่ตอบคำถามนอกขอบเขตแพลตฟอร์ม หากคำถามเกินความสามารถของ AI ระบบจะแนะนำให้ดู FAQ แทน เพราะ V1 ยังไม่มีระบบ support ticket จริง ประวัติการสนทนาอยู่ในระดับ session เดียวกับ AI-002/AI-003 เมื่อ AI ตอบไม่ได้ติดต่อกันครบ 3 ครั้ง หรือผู้ใช้พิมพ์ขอคุยกับเจ้าหน้าที่โดยตรง เช่น "คุยกับแอดมิน" หรือ "ติดต่อเจ้าหน้าที่" ระบบจะเสนอปุ่มสำหรับส่งต่อให้ Admin โดยไม่ต้องรอให้ AI ตอบพลาดครบสามครั้งก่อน กดแล้วบทสนทนานั้นเปลี่ยนสถานะเป็น ESCALATED และเข้าคิวรอ Admin สถานะของบทสนทนามี 3 ค่า คือ AI_ONLY เป็นค่าเริ่มต้น, ESCALATED และ RESOLVED ระบบต้องตรวจเงื่อนไขการส่งต่อซ้ำที่ฝั่ง server ไม่เชื่อคำที่ client ส่งมา และการกดซ้ำหรือกดตอนที่ส่งต่อไปแล้วต้องไม่สร้างคิวซ้ำ เมื่อบทสนทนาอยู่ในสถานะ ESCALATED ระบบจะไม่เรียก AI อีก ข้อความของผู้ใช้จะถูกส่งถึง Admin โดยตรง ฝั่ง Admin เห็นคิวบทสนทนาที่รออยู่ กดรับผิดชอบบทสนทนาโดยบันทึกไว้ที่ assigned_admin_id ตอบกลับ และกดปิดเป็น RESOLVED ได้ ถ้าผู้ใช้พิมพ์เข้ามาอีกหลังปิดไปแล้ว บทสนทนาต้องกลับมาเป็น

ID	ข้อกำหนด	เกณฑ์การยอมรับ
		ESCALATED อีกครั้งโดยอัตโนมัติ ทั้งสองฝั่งเห็นข้อความใหม่แบบ realtime และบทสนทนาถูกบันทึกลงฐานข้อมูล ผู้ใช้จึงกลับมาอ่านย้อนหลังได้แม้ปิดหน้าต่างไปแล้ว
AI-002	AI Price Estimator (ฝั่งประมูล) (Optional)	พีเจอร์เสริม — ระหว่างทำขั้นตอน AUC-001 ผู้ขายสามารถขอให้ AI ประเมินราคาได้ โดย model ที่รับรูปภาพได้จะวิเคราะห์รูปที่อัปโหลด ร่วมกับหมวดหมู่ที่เลือกและสภาพสินค้าที่ระบุไว้ แล้วคืนค่าราคาเริ่มต้นที่แนะนำ พร้อมช่วงราคาปิดประมูลโดยประมาณและเหตุผลสั้นๆ ประกอบ ผลลัพธ์นี้เป็นเพียงคำแนะนำเท่านั้น: ราคาเริ่มต้นและ reserve ยังคงเป็นช่องที่ผู้ขายกรอกเอง (AUC-002/AUC-003) การ publish ไม่จำเป็นต้องยอมรับคำแนะนำนี้เลยก็ได้ และ UI จะระบุตัวเลขเหล่านี้ว่าเป็น “ค่าประเมิน” ไม่ใช่การตีราคาหรือการรับประกันใดๆ แต่ละ draft จะจำกัดจำนวนครั้งที่ขอประเมินได้ในจำนวนครั้งที่น้อย เพื่อควบคุมต้นทุนและป้องกันการใช้งานผิดวัตถุประสงค์
AI-003	AI Negotiator (ฝั่ง e-commerce) (Optional)	พีเจอร์เสริม — ที่หน้า PROD-004 ผู้ซื้อสามารถเสนอราคาและจำนวนที่ต้องการซื้อสำหรับสินค้าที่มีการตั้ง negotiation floor ไว้ (PROD-006) ระบบ negotiator จะพิจารณาราคาที่เสนอเทียบกับราคาต่ำสุดที่เป็นความลับ, ราคาปัจจุบัน และสต็อกคงเหลือ แล้วตอบกลับทันทีเป็น ACCEPT, COUNTER (พร้อมราคาต่อรองที่ระบบเสนอกลับ) หรือ REJECT — โดยจะไม่มีทางอนุมัติราคาที่ต่ำกว่าราคาต่ำสุดที่ตั้งไว้เด็ดขาด การตัดสินใจเกิดขึ้นทันที ส่วนสิ่งที่ตามมาคือช่วงเวลาแยกต่างหากสำหรับดำเนินการต่อ เมื่อได้รับ ACCEPT แล้ว ระบบจะปลดล็อกราคาที่ต่อรองได้ในเวลาสั้นๆ (15 นาที) ใช้ได้ครั้งเดียว ผูกกับผู้ซื้อสินค้า และจำนวนที่เสนอไว้นั้นๆ โดยผู้ซื้อต้อง checkout ตามปกติ (CART-004) ให้เสร็จภายในช่วงเวลานี้ — การต่อรองราคาเองไม่ได้ทำให้การซื้อสำเร็จ เหตุผลที่ต้องมีเวลามหดยาคือเพื่อป้องกันไม่ให้ผู้ซื้อสะสมข้อเสนอที่ accept แล้วจากสินค้าหลายรายการไว้ใช้ทีหลัง และป้องกันไม่ให้ข้อเสนอเก่าที่ค้างอยู่ยังใช้ได้หลังจากผู้ขายแก้ไขราคา/floor ไปแล้ว (PROD-002) สต็อกยังคงถูกตรวจสอบและตัดตอน checkout ตามปกติ (PROD-005) ดังนั้นข้อเสนอที่ accept แล้วก็ยังอาจล้มเหลวได้หากสินค้าหมดไปก่อน การเสนอราคามี cooldown 5 นาทีก่อนเสนอครั้งถัดไปได้ และจำกัดสูงสุด 3 ครั้งต่อ (ผู้ซื้อ, สินค้า) ต่อ 24 ชั่วโมง — นับทุกครั้งที่เสนอราคาเป็น 1 attempt เท่ากันหมด ไม่ว่าผลจะเป็น ACCEPT/COUNTER/REJECT และไม่ว่าจะ checkout จริงหรือปล่อยให้ token หหมดอายุก็ตาม เพื่อปิดช่องให้ผู้ซื้อไล่เดาหา floor แบบไม่มีต้นทุน (ตัวเลข 5 นาที/3 ครั้งปรับได้ตามความเหมาะสม ไม่ตายตัว)

4.8 การบริหารจัดการระบบ (Admin)

ID	ข้อกำหนด	เกณฑ์การยอมรับ
ADM-001	ควบคุมดูแลประมูล	Admin ยกเลิกประมูลที่ไม่เหมาะสมได้ พร้อมบันทึกการกระทำและเหตุผลไว้
ADM-002	จัดการผู้ใช้	Admin ระบุ/เปิดใช้งานผู้ใช้ใหม่ได้ ผู้ใช้ที่ถูกระบุจะ login ไม่ได้ สร้างประมูลหรือลงขายสินค้าไม่ได้ ประมูลไม่ได้ เพิ่มลงตะกร้าไม่ได้ และ checkout ไม่ได้
ADM-003	จัดการหมวดหมู่	Admin สร้าง, แก้ไข, เปิด และปิดใช้งานหมวดหมู่ได้ หมวดหมู่ที่ถูกใช้งานอยู่แล้วจะถูกปิดใช้งาน ไม่ใช้ลบทิ้งถาวร
ADM-004	บันทึก Audit	การกระทำสำคัญจะถูกบันทึกไว้ ประกอบด้วย ผู้ดูแลระบบที่ทำ, ประเภทการกระทำ, เป้าหมาย, หมายเหตุ และเวลา ส่วนการคืนสถานะและการตรวจสอบรายงานยังเลื่อนออกไปก่อน
ADM-005	ควบคุมดูแลรายการสินค้า	Admin ปิดการขายสินค้าที่ไม่เหมาะสมได้ โดยเปลี่ยนสถานะสินค้าเป็น SUSPENDED ซึ่งเป็นสถานะที่สงวนไว้ให้ Admin เท่านั้น ผู้ขายเปลี่ยนออกจากสถานะนี้เองไม่ได้ (ดู PROD-002) มีเพียง Admin เท่านั้นที่เปิดการขายกลับได้ โดยระบบจะคืนสถานะเป็น ACTIVE หรือ OUT_OF_STOCK ตามจำนวนสต็อกคงเหลือ ณ ขณะนั้น ระหว่างที่สินค้าอยู่ในสถานะ SUSPENDED การที่ผู้ขายเติมสต็อกจะไม่ทำให้สถานะเปลี่ยนเป็น ACTIVE โดยอัตโนมัติ (อ้างอิงจาก PROD-005) ทุกครั้งที่ปิดหรือเปิดการขายต้องบันทึก

ID	ข้อกำหนด	เกณฑ์การยอมรับ
		เหตุผลไว้ เหมือนกับ ADM-001 การปิดการขายจะปิดกันคำสั่งซื้อใหม่ (ค้นหาไม่เจอในหน้าสาธารณะ เพิ่มลงตะกร้าและ checkout ไม่ได้) แต่ไม่ยกเลิกคำสั่งซื้อที่จ่ายเงินแล้ว (PAID)
ADM-006	ภาพรวมคำสั่งซื้อ	Admin ดูรายการคำสั่งซื้อ e-commerce ทั้งหมดแบบแบ่งหน้าและดูอย่างเดียว (buyer, seller, สถานะ, ยอดรวม) เพื่อใช้สนับสนุน/ตรวจสอบ ใน V1 ยังไม่มีฟังก์ชันแก้ไขหรือคืนเงิน — การจัดการข้อพิพาทเลื่อนออกไปก่อนอย่างชัดเจน สอดคล้องกับการเลื่อนคิวรายงานของโมดูล Auction

5. โครงสร้างข้อมูล, API และ Event Contract

5.1 โครงสร้างข้อมูลหลัก

โดเมน	Entity ปัจจุบันใน V1 / field สำคัญ
Identity	users, user_profiles, auth_accounts (provider GOOGLE/LINE), user_sessions, two_factor_codes, password_reset_tokens , trusted_devices (เก็บ hash ของ token ประจำ browser, วันหมดอายุ, เวลาที่ใช้ล่าสุด และเวลาที่ถูกยกเลิก — AUTH-007) (hashed token, expiry, used flag — รูปแบบเดียวกับ two_factor_codes) (เก็บ OTP แบบ hash, วันหมดอายุ, สถานะการใช้งาน — ไม่มี secret แบบระยะยาวเหมือน TOTP); role ที่บันทึกคือ USER/ADMIN
Shared	categories ใช้ร่วมกันทั้งสองโมดูล ไม่แยกขอบเขตตามโมดูล; รายการในแต่ละโดเมนจะอ้างอิงได้เฉพาะหมวดหมู่ที่ active เท่านั้น
Auction	auctions, auction_images, bids, auction_extensions, auction_events; สถานะ DRAFT/SCHEDULED/ACTIVE/SOLD/UNSOLD/CANCELLED
E-commerce	products, product_images (มี negotiation_floor แบบส่วนตัวได้ ตาม PROD-006 และ quantity_discount_min_qty/quantity_discount_percent ตาม PROD-007), carts, cart_items, offers (buyer, product, จำนวน, offer_amount, decision ACCEPTED/COUNTERED/REJECTED, counter_amount, expires_at), orders (แต่ละ order อ้างอิง payment_transaction ผ่าน checkout_session_id ร่วมกัน), order_addresses (...), order_items, payment_transactions (จำลอง, unique ต่อ checkout_session_id — 1 payment ต่อ 1 ครั้งที่กดจ่าย ไม่ใช่ 1 payment ต่อ 1 order), conversations (unique ต่อ product_id+buyer_id+seller_id), messages (conversation_id, sender_id, body, created_at, read_at), shipments, shipment_events; ; order_items มี product_id และ auction_id เป็น nullable ทั้งคู่ โดย 1 บรรทัดผูกกับอย่างใดอย่างหนึ่งเสมอ และมี unique index บน auction_id เพื่อให้ประมวล 1 รายการถูกจ่ายได้ครั้งเดียว (CART-004); conversations มี product_id และ auction_id เป็น nullable ทั้งคู่ในรูปแบบเดียวกัน พร้อม unique อีกชุดคือ auction_id+buyer_id+seller_id (CHAT-004); users มีคอลัมน์ auto_reply_message สำหรับข้อความตอบกลับอัตโนมัติของผู้ขาย (CHAT-004) สถานะสินค้า ACTIVE/INACTIVE/OUT_OF_STOCK/REMOVED/SUSPENDED (SUSPENDED สงวนไว้ให้ ADM-005 เท่านั้น ผู้ขายเปลี่ยนออกเอง ไม่ได้); สถานะออเดอร์ PENDING/PAID/CANCELLED; สถานะการจัดส่ง PROCESSING/SHIPPED/IN_TRANSIT/DELIVERED/CANCELLED
Engagement	watchlists, product_watchlists (composite key user_id+product_id แบบเดียวกับ watchlists แต่แยกตาราง เพราะสินค้ากับประมวล อยู่คนละตาราง) auction_participants (JOINED/LEFT), notifications (ประเภทฝั่งประมวลและออเดอร์/จัดส่ง พร้อม reference กลับไปยัง auction/order/conversation ที่เกี่ยวข้อง เพื่อให้ client รู้ว่าต้อง refetch อะไรตาม §5.2), admin_actions, support_chat_sessions/support_chat_messages โดย support_chat_sessions มี status เป็น AI_ONLY/ESCALATED/RESOLVED และ assigned_admin_id ขึ้นกับผู้สิทธิ์ ADMIN ที่รับผิดชอบบทสนทนานั้น ส่วน role ของข้อความเพิ่มค่า ADMIN นอกเหนือจาก USER/ASSISTANT (AI-001) (บทสนทนา Customer Service AI ระดับ session ไม่บังคับเก็บข้ามการ login ใหม่), ai_requests (ประเภท PRICE_ESTIMATE หรือ NEGOTIATION จาก AI-002/AI-003 ที่เป็น Optional ใช้เป็น log สำหรับ rate-limit เท่านั้น)
Integrity	Bid ที่ยอมรับแล้วมี client_request_id และ (auction_id, sequence_no) ที่ไม่ซ้ำกัน; Watchlist ใช้ composite key ของ user/auction; social account ไม่ซ้ำกันตาม provider/account ID และ user/provider (รวม LINE ด้วยแล้ว); Order ไม่ซ้ำกันตาม (checkout_session_id, seller_id) payment_transactions ไม่ซ้ำกันตาม checkout_session_id (unique) — orders หลายใบในเซสชันเดียวกันขึ้นไป payment_transaction แยกเดียวกัน ไม่สร้างแยกต่อ order ทำให้การ checkout แบบหลายผู้ขายครั้งเดียวได้ Order แยกตามผู้ขายภายใต้ session เดียวกัน; การติดสต็อกถูกป้องกันด้วย transaction เกี่ยวกับการสร้างออเดอร์เพื่อไม่ให้ขายเกินสต็อก; Offer ที่ accept แล้วใช้ได้ครั้งเดียว การใช้งานหรือหมดอายุจะป้องกันการใช้ซ้ำ และราคาที่ต้องรองได้จะต้องไม่ต่ำกว่า negotiation floor ที่เป็นความลับของสินค้านั้นเด็ดขาด (PROD-006); การสนทนา (conversation)

โดเมน	Entity ปัจจุบันใน V1 / field สำคัญ
	ไม่ซ้ำกันตาม (product_id, buyer_id, seller_id) — เปิดแชทซ้ำกับผู้ขายเดิมสำหรับสินค้าเดิมจะกลับไปใช้ thread เดิมเสมอ สำหรับห้องสนทนาฝั่งประมูลใช้ unique อีกชุดคือ (auction_id, buyer_id, seller_id) แยกจากกัน; การชำระเงินของผู้ชนะประมูลกับการจ่ายค่าด้วย unique บน order_items.auction_id ซึ่ง Postgres ถือว่าค่า NULL ต่างกันเสมอ ข้อจำกัดนี้จึงมีผลเฉพาะบรรทัดที่เป็นประมูล ไม่กระทบบรรทัดสินค้าปกติ

5.2 REST และ WebSocket ขั้นต่ำที่ต้องมี

- กลุ่ม REST endpoint: /auth (รวม /auth/2fa/verify, /auth/2fa/resend, /auth/line/callback), /auth/forgot-password, /auth/reset-password, /users/me, /categories, /auctions, /auctions/:id/bids, /auctions/drafts/:id/price-estimate, /watchlist, /notifications, /products, /products/:id/offers, /cart, /orders, /orders/:id/shipment, /support/chat, /products/:id/conversations, /conversations/:id/messages และ /admin, /auctions/:id/conversations (เปิดห้องสนทนาฝั่งประมูล — CHAT-004), /users/me/auto-reply (ข้อความตอบกลับอัตโนมัติของผู้ขาย — CHAT-004), /products/:id/watchlist และ /watchlist/products (ติดตามสินค้าในร้าน — WAT-001/002), /support/chat/:sessionId/escalate และ /admin/support/sessions (ส่งต่อบทสนทนาให้ Admin — AI-001)
- Event ขั้นต่ำที่ต้องมี: auction:started, auction:bid, auction:extension, auction:ended, order:status_changed, message:sent และ notification:created, support:message และ support:inbox_updated (ข้อความและคิวบสนทนาฝั่ง Admin — AI-001)

6. ข้อกำหนดด้านความปลอดภัยและคุณภาพ

- ตรวจสอบข้อมูล input จากภายนอกทุกจุด; บังคับใช้การตรวจสอบสิทธิ์ที่ฝั่ง server ไม่ใช่แค่ที่ UI; hash รหัสผ่าน, refresh token, รหัส OTP และ reset token ก่อนบันทึกลงฐานข้อมูล; ห้าม log รหัส OTP หรือ reset token แบบข้อความล้วนเด็ดขาด
- ห้ามเปิดเผยราคา reserve ของประมูล, negotiation floor ของสินค้า, ข้อมูลส่วนตัวของผู้ประมูล หรือตะกร้า/คำสั่งซื้อของผู้ใช้คนอื่นเด็ดขาด จำกัดความเร็ว (rate-limit) การ login, การตรวจสอบ/ขอส่ง OTP ใหม่, การส่งบิต, การขอประเมินราคา AI และการเสนอราคาต่อรอง; ส่ง error ที่เป็นประโยชน์กลับไปโดยไม่หลุดรายละเอียดการ implement ออกไป
- ใช้ transaction หรือกลไกป้องกัน concurrency ที่เทียบเท่ากันสำหรับการประมูล การตัดสต็อก และการสร้างออเดอร์แบบหลายผู้ขาย; event บิตที่ยอมรับแล้วและออเดอร์ที่ PAID จะเกิดขึ้นได้ก็ต่อเมื่อมาจากผลลัพธ์ transaction ที่ถูกต้องเท่านั้น
- ระบุให้ชัดเจนใน UI ว่าข้อมูลการชำระเงินและการจัดส่งทั้งหมดเป็นข้อมูลจำลอง (เช่น ใส่ badge “Test / Simulated” ค้างไว้ในหน้า checkout และหน้าต่างติดตามพัสดุ) เพื่อไม่ให้เข้าใจผิดว่าเป็นธุรกรรมจริง
- Maildev เป็นตัวดักอีเมลสำหรับ development/test/CI เท่านั้น และห้ามเข้าถึงได้จาก production network เด็ดขาด เพราะหน้า dashboard ของมันเปิดเผยรหัส OTP ทุกตัวโดยไม่มีการยืนยันตัวตนใดๆ ดังนั้น production จึงต้องส่ง AUTH-007 ผ่าน SMTP relay จริงเสมอ ผ่าน interface เดียวกัน (ดู §3)
- ทุก request ที่เรียก API ต้องผ่านการตรวจสอบ token นั้นซ้ำโดย NestJS เสมอ (AUTH-008)
- Customer Service AI Chatbot (AI-001) ตอบเฉพาะคำถามเกี่ยวกับแพลตฟอร์มเท่านั้น ไม่ตอบคำถามนอกขอบเขต ไม่เปิดเผยข้อมูลของผู้ใช้คนอื่น และไม่ทำหน้าที่เป็นช่องแชทโดยตรงระหว่างผู้ซื้อ-ผู้ขาย ทุกคำขอถูก rate-limit เช่นเดียวกับ AI-002/AI-003
- บันทึก log การกระทำของ Admin ที่เกี่ยวข้องกับความปลอดภัย หน้าจอต้อง responsive รองรับทั้ง desktop, tablet และ mobile
- การส่งมอบฝั่ง E-commerce เน้นเช่นเดียวกับขอบเขตที่เลือกของโมดูล Auction คือ การเชื่อมต่อ Next.js, automated test coverage และการทำให้ระบบเสถียรก่อนปล่อยจริง โดยรันผ่าน Docker workflow เดียวกัน
- ห้ามเปิดเผยเนื้อหาการสนทนาให้ผู้ใช้นอกเหนือจากคู่สนทนา (buyer/seller) เด็ดขาด รวมถึง Admin ใน V1 (ไม่มีระบบตรวจสอบ/moderation เนื้อหาแชท)

7. หัวข้อสำคัญที่ต้องทดสอบให้ผ่าน

7.1 ฟังก์ชันประมวล

1. User A สมัครสมาชิกแบบ local หรือ login ผ่าน Google หรือ Line จากนั้นกรอกรหัส 2FA ที่ส่งทางอีเมลให้ครบตามที่บังคับไว้
2. ความถูกต้องของการแนะนำจาก AI เช่น User A เปิด Customer Service AI Chatbot ถามวิธีการตั้งค่า reserve ของประมูล และได้รับคำตอบที่ถูกต้องตรงประเด็น (ทดสอบ AI-001) จากนั้นสร้าง, preview และ publish ประมูลแบบ scheduled
3. User B กด watch ประมูลนั้นและเข้าร่วม lobby
4. เมื่อถึงเวลาเริ่ม ประมูลจะเปลี่ยนเป็น Active โดยไม่ต้อง refresh หน้าเอง
5. User B และ C ส่งบิตแข่งกันแบบเรียลไทม์และถูกต้องตามเงื่อนไข
6. บิตที่เข้ามาในช่วง 2 นาทีสุดท้ายทำให้ประมูลต่อเวลาได้ถูกต้อง
7. ผู้ที่เคยนำอยู่ก่อนหน้าจะได้รับการแจ้งเตือน Outbid
8. เมื่อจบประมูล ระบบสรุปผล Sold หรือ Unsold ได้ถูกต้อง และสร้างการแจ้งเตือนให้ทั้งผู้ชนะและผู้ขาย
9. Admin จัดการผู้ใช้/หมวดหมู่ และการกระทำนั้นถูกบันทึกไว้; Admin ยกเลิกประมูลที่ไม่เหมาะสมได้พร้อมระบุเหตุผล
10. flow หลักทั้งหมดรันผ่าน Docker และผ่าน E2E test ที่สำคัญ

7.2 ฟังก์ชัน E-commerce

1. ผู้ขายลงขายสินค้าพร้อมหมวดหมู่, ราคา, รูปภาพ, จำนวนสต็อก และอาจตั้ง negotiation floor แบบเป็นความลับด้วยก็ได้
2. ผู้ซื้อค้นหาและกรองแคตตาล็อกตามหมวดหมู่ แล้วเพิ่มสินค้าจากผู้ขาย 2 รายที่ต่างกันลงตะกร้า
3. ผู้ซื้อ checkout ด้วยที่อยู่จัดส่งเริ่มต้นในโปรไฟล์ ก่อนที่ราคาต่อรองจะหมดอายุ ตะกร้าจะถูกแยกเป็น 2 Order อัตโนมัติ แยกตามผู้ขายแต่ละราย ภายใต้ checkout session เดียวกัน
4. ระบบชำระเงินจำลองสินค้า SUCCEEDED ออเดอร์ทั้งสองเปลี่ยนเป็น PAID และตัดสต็อกแบบ atomic ที่ละรายการ
5. ผู้ขายแต่ละรายอัปเดตสถานะจัดส่งของออเดอร์ตัวเอง ผู้ซื้อได้รับการแจ้งเตือน Shipment Update ทุกขั้นตอน และติดตามทั้งสองออเดอร์จนถึงสถานะ DELIVERED
6. ผู้ซื้อดู Order History ของทั้งสองผู้ขายรวมกัน ผู้ขายแต่ละรายก็ดู Order History ของออเดอร์ที่ตัวเองได้รับเช่นกัน
7. ความถูกต้องของการแนะนำจาก AI เช่น ผู้ซื้อเปิด Customer Service AI Chatbot ถามสถานะออเดอร์ล่าสุดของตัวเอง และได้รับคำตอบที่ถูกต้องตรงกับข้อมูลจริง (ทดสอบ AI-001)
8. Admin ปิดการขายสินค้าที่ไม่เหมาะสมพร้อมระบุเหตุผล และการกระทำนั้นถูกบันทึกไว้ จากนั้นผู้ขายพยายามเปิดการขายกลับเอง แก้ไขข้อมูลสินค้า และลบสินค้านั้น ทั้งสามอย่างต้องถูกปฏิเสธ และการเดิมสต็อกต้องไม่ทำให้สินค้ากลับมาขายได้เอง